Summary of Book6 Sans 401

Course Outline
• Network Security and Cloud
Essentials
• Defense-in-Depth
• Vulnerability Management and
Response
• Data Security Technologies
• Windows and Azure Security
• Linux, AWS, and Mac Security

Linux, AWS, and
Mac Security

Windows: From Desktop → to Server
How Windows Started

Windows was originally made for one person using one computer.

Example: Someone writing a report or browsing the internet.

It was mainly a desktop operating system.

What Changed?

As companies grew:

Many people needed to work together

They needed to share files and printers

They needed one system to manage everything

So Microsoft created Windows Server.

Now Windows Has:

Windows Desktop → for personal computers

Windows Server → for companies, multiple users, file sharing, Active Directory, etc.

So Windows evolved from:

Single-user desktop ➜ Multi-user server

UNIX / Linux: From Server → to Desktop

4

How UNIX Started

UNIX was created in 1969.

It was designed for many users at the same time.

It ran on big servers, not personal computers.

It managed:

Multiple users

Background processes

System resources

Then What Happened?

UNIX became very stable and powerful.

Companies created their own UNIX versions.

Later, someone created a free version called Linux and shared it with the world.

Linux eventually became:

A server OS

A desktop OS (like Ubuntu)

The foundation of Android

The core of most internet servers today

So UNIX/Linux evolved from:

Multi-user server ➜ Personal desktop

Why Security People Must Know Both

Today:

Windows and Linux machines are connected on the same networks.

They can attack each other.

Each has different:

Strengths

Weaknesses

Security risks

If you're learning cybersecurity (like SEC401 topics you're studying), you must understand both.

Linux has many versions (distros).

Each distro is built for a specific goal:

Desktop

Server

Cloud

Security

Ubuntu = beginner-friendly

Debian = stable

Fedora = more advanced & secure-focused

It Started With One Person

In the early 1990s, a Finnish programmer named
Linus Torvalds
started a hobby project.

He wanted to improve a small teaching operating system called:

MINIX

MINIX was used in universities to teach how UNIX works.

What Did He Do?

He rewrote the system from scratch

Designed it for Intel 386 computers

Created one of the first UNIX-like systems for personal computers

And here's the important part:

He made it free and open-source
He shared the source code with the world
He didn't ask for money

This was revolutionary.

What Is the Kernel?

At the center of Linux is something called the kernel.

The kernel:

Connects hardware and software

Manages CPU, memory, and devices

Is the "core" of the operating system

So when we say "Linux," we technically mean:

The Linux kernel

The GNU Project Connection

Before Linux, there was already:

GNU Project

GNU was creating:

Free tools

Compilers

Utilities

Applications

But it was missing one thing:

❌ A working kernel

When Torvalds released Linux:

✔ GNU got the missing kernel
✔ Linux got powerful GNU tools

Together they formed a complete operating system.

That's why the correct name is:

GNU/Linux

(But most people just say "Linux.")

Why Linux Became So Big

After Linux was released:

Programmers around the world started improving it

Thousands of developers contributed

It became:

More stable

More powerful

More secure

Today:

Most web servers run Linux

Most cloud infrastructure runs Linux

Android is based on Linux

Supercomputers run Linux

And Linus Torvalds still oversees the Linux kernel today.

What Is WSL?

Windows Subsystem for Linux (WSL) is a feature in Windows 10 and Windows 11 that lets you run Linux inside Windows.

So instead of:

Installing Linux separately ❌

Or dual booting ❌

Or installing a heavy virtual machine ❌

You can just run Linux directly in Windows ✅

What Is WSL 2?

WSL 2 is the improved version.

It:

Uses a real Linux kernel

Runs inside a lightweight virtual machine

Has better:

File system performance

Compatibility

Speed

It uses virtualization, but it's much lighter than traditional tools like VirtualBox.

What Can You Install?

From the Microsoft Store, you can install Linux distributions like:

Ubuntu

Kali Linux

Debian

openSUSE

You choose one, install it, and start using Linux commands in a terminal window.

What Does This Allow Windows Users To Do?

With WSL, Windows users can:

✔ Run Linux commands (ls, grep, apt, etc.)
✔ Use Bash shell
✔ Install Linux tools
✔ Develop software for Linux
✔ Run security tools (like Kali tools)
✔ Use Docker and containers

All without leaving Windows.

Why Is WSL Important?

Especially for:

Developers

Cybersecurity students (like the topics you're studying)

Cloud engineers

DevOps professionals

It allows you to:

Work in both Windows and Linux environments on one machine.

Very powerful for learning and security labs.

Linux Vulnerabilities (High-Risk Examples)

Big software projects (like Linux) sometimes have bugs.
If a bug allows attackers to break security → it's called a vulnerability.

Attackers can exploit vulnerabilities to:

Run malicious code

Steal data

Gain root/admin access

Control systems remotely

Let’s look at 3 major examples in simple terms.

1 Shellshock (2014)
4

Official name: CVE-2014-7169
Affected: Bash shell

What Was the Problem?

Bash (Linux command shell) incorrectly handled environment variables.

An attacker could:

Hide malicious commands inside an environment variable

When Bash started → it would automatically execute that hidden code

Why Was It Dangerous?

✔ Could be triggered remotely
✔ Affected web servers
✔ Allowed remote code execution
✔ Could lead to privilege escalation

It caused major panic in 2014 because many internet servers were vulnerable.

2 SambaCry (2017)
4

Official name: CVE-2017-7494
Affected: Samba service

Samba allows Linux machines to share files using the SMB protocol (Windows file sharing).

What Was the Problem?

If Samba had a writable file share:

An attacker could upload a malicious file

Then execute it remotely

And gain root privileges

Why Is It Called “Linux’s WannaCry”?

Because it was similar in impact to:

Windows’ EternalBlue exploit

WannaCry ransomware

It allowed remote code execution over the network.

3 Baron Samedit (2021)
4

Official name: CVE-2021-3156
Discovered by: Qualys research team
Affected: sudo

What Is sudo?

sudo lets normal users run commands as root (administrator).

What Was the Problem?

A heap-buffer overflow bug in sudo allowed:

Any normal user

To gain root access

Without knowing the password

Even worse:

The bug existed for almost 10 years

What Is a Heap Buffer Overflow?

The heap is memory used for dynamic data.

If a program:

Writes more data than allocated

It can overwrite memory

Attackers can inject malicious code

That’s what happened here.

Linux Vulnerabilities (2): Sudo Bypass

Sudo Bypass (CVE-2019-14287)

Official ID: CVE-2019-14287
Discovered: October 2019
Affected: sudo

First: What Is sudo?

In Linux, sudo means:

Run a command as another user (usually root).

Example:

sudo apt update

Normally:

Regular users = limited privileges

Root (user ID 0) = full control of system

sudo is controlled by a configuration file called sudoers policy.

What Was the Bug?
4

Sudo allows running commands as a specific user by providing:

Username
OR

User ID (UID)

Example:

sudo -u username command

The Problem

If a user had permission to run commands as any user except root, they could:

Use this trick:

sudo -u#-1 id -u
What does this mean?

-u#-1 = run as user ID -1

Linux internally converts -1 → to 0

User ID 0 = root

So even if sudoers said:
❌ "You cannot run as root"

The system would accidentally:
✔ Convert -1 → 0
✔ Run command as root

That means full system control.

Why Did This Happen?

Before running the command, sudo calls system functions:

setresuid(2)

setreuid(2)

These are system calls used to change user IDs.

Because sudo itself already runs with root privileges internally,
these functions returned:

0 (which is root)

So the input -1 was not handled correctly.

When Could It Be Exploited?

This vulnerability could only be exploited if:

✔ The user already had some sudo privileges
✔ The sudo policy allowed running commands as arbitrary user IDs

So it wasn't a remote exploit —
it was a privilege escalation vulnerability.

Physical Filesystem in Linux (Easy Explanation)

One System… But Many Pieces

In Linux, the filesystem looks like one big tree:

```
/
 ├── /usr
 ├── /var
 ├── /home
 ├── /tmp
```

But in reality

It is made of multiple physical disk partitions.

What Is a Partition?

4

A partition is:

A physical section of a hard disk.

Example:

One partition for /

One partition for /usr

One partition for /var

One partition for /home

During boot:

Linux mounts each partition

It attaches them to the directory tree

The user sees one unified system

What Is Mounting?

Mounting means:

Connecting a disk partition to a directory.

Example:

A partition is mounted at /var

Another is mounted at /usr

All mount info is stored in:

/etc/fstab

This file:

Lists partitions

Defines mount points

Defines security options

Why Is Partitioning Important for Security?

This is a key Linux security design feature:

Security can be built into the installation process.

Each partition can have different security options.

Example mount options:

noexec → No program execution allowed

nosuid → Ignore set-UID programs

nodev → No device files allowed

ro → Read-only

This means:

/usr can be read-only

/tmp can block executable files

/home can have different restrictions

Why Not Use One Big Partition?

Some admins use:

One partition for entire disk

This is a bad idea.

Here’s why

1 Prevent Denial of Service (DoS)

Example:

Logs are stored in /var

If /var fills up with logs

Only /var stops working

But if everything is one partition:

The entire system disk fills up

The whole OS crashes

So separating /var protects the system.

2 Easier Backups

/usr rarely changes → No need daily backup

/home changes daily → Needs frequent backup

Partitions make backups easier and faster.

3 Protect Critical System Files

Attackers who break into a system often try to:

Install a rootkit

Replace system binaries in:

/usr/bin

/usr/sbin

If /usr is:

Mounted as read-only

Or protected with strict permissions

It becomes much harder to modify system files.

Linux Unified Key Setup (LUKS)

What is LUKS?

LUKS (Linux Unified Key Setup) is the standard system used in Linux to encrypt hard disks.
Encryption means protecting your data so nobody can read it without the correct password.

Think of it like locking your hard drive with a strong digital lock.

Why LUKS is useful

LUKS solves several problems in disk encryption:

Standard format

Many old encryption tools used different formats.

LUKS created one standard format that works across many Linux systems.

Compatibility

Because it is standard, different Linux distributions and tools can read the encrypted disk.

Multiple passwords

LUKS allows several passwords (keys) for the same encrypted disk.

Example:

Admin password

User password

Header information

LUKS stores important encryption information in the partition header.

This allows the encrypted disk to be moved to another system and still work.

SUID and SGID Programs (Linux Security Risk)
What are SUID and SGID?

In Linux, files can have special permissions:

SUID (Set User ID)

SGID (Set Group ID)

These permissions allow a program to run with the privileges of the file owner or group, not the user who runs it.

Example:

A normal user runs a program

The program has SUID root

The program runs with root privileges

So the user temporarily gets higher privileges.

Simple Example

A good example is the Linux command:

passwd

Normal users cannot modify system password files.
But passwd has SUID root, so when you run it:

It runs with root privileges

It allows you to change your password only

Without SUID, this would not work.

Why SUID/SGID is called a "Double-Edged Sword"

They are necessary but dangerous.

1 Necessary

Linux systems need them to perform some tasks.

Examples:

passwd

sudo

ping

These programs need higher privileges.

2 Dangerous

Attackers often exploit vulnerable SUID programs.

Example attacks:

Malicious SUID shell

Buffer overflow in SUID program

Privilege escalation

Example scenario:

Attacker places a SUID root shell on a USB/CD

A user runs it

The attacker gets root access without password

Golden Rules for Administrators

Rule 1

Keep a list of all SUID/SGID programs on the system when it is first installed.

Rule 2

Regularly check the system for new SUID/SGID programs.

If you find a new one, it may mean:

Malware

Backdoor

System compromise

How to Find SUID and SGID Files

Linux administrators use the find command.

Find SUID files
find / -perm +4000

Meaning:

Search the entire system /

Find files with SUID permission

Find SGID files
find / -perm +2000

Meaning:

Search for files with SGID permission

File Ownership in Linux

In Linux, every file and directory has two types of ownership:

User (Owner) – the person who owns the file

Group – the group that can access the file

Changing permissions (rwx) is not enough for security.
Sometimes you must also change who owns the file.

That is why Linux provides two commands:

chown → change owner

chgrp → change group

chown – Change File Owner

The chown command changes the user who owns a file or directory.

Example:

chown user1 /home/user1

Meaning:

The owner of /home/user1 becomes user1

Usually only the root user (UID 0) can run chown.

This prevents normal users from changing file ownership.

chgrp – Change Group Owner

The chgrp command changes the group associated with a file.

Example:

chgrp support /home/user1

Meaning:

The file /home/user1 now belongs to the support group

Members of the support group may get access depending on permissions.

Changing Owner and Group Together

You can change both user and group at the same time using chown.

Format:

chown owner:group filename

Example:

chown user1:support /home/user1

Meaning:

Owner → user1

Group → support

UID and GID

Each user and group has a number ID:

UID → User ID

GID → Group ID

Linux can use names or numbers.

Example:

chown 1001:1002 file.txt

or

chown user1:support file.txt

Both work.

Security Risk (Important)

If normal users were allowed to freely use chown, they could:

Change the owner of a program to root

Enable SUID

Run the program and get root privileges

This would allow privilege escalation.

Because of this risk:

Most systems allow only root (UID 0) to run chown.

If ownership changes, Linux removes SUID/SGID bits automatically.

Disk Quota Problem

If users can change file ownership:

User1 could create a large file

Then run:

chown user2 file.txt

Now the disk usage counts against user2's quota, not user1.

This is another reason why chown is restricted.

✅ Short Summary

Command	Purpose
chown	Change file owner

chgrp Change group owner
chown user:group file Change both owner and group
UID 0 Root user who can run chown

User Accounts and Groups in Linux

1 Usernames and Passwords

Every Linux user has:

Username

Password

Both are case sensitive.

Example:

User1

user1

These are two completely different users.

✅ Best practice:
Use lowercase usernames only to avoid confusion.

2 Usernames vs IDs (UID / GID)

Humans use names, but Linux internally uses numbers.

Item	Meaning
UID	User ID
GID	Group ID

Example:

Username	UID
user1	1001

Linux actually tracks files using UID and GID, not the names.

3 Important System Files

Linux stores account information in two main files.

/etc/passwd

Contains:

username

UID

GID

home directory

default shell

Example entry:

user1:x:1001:1001:/home/user1:/bin/bash
/etc/shadow

Contains encrypted passwords.

This file is protected and readable only by root.

4 What Happens if a User is Deleted?

Example:

User user1 creates a file.

Admin deletes user1.

The file still exists.

Now Linux shows the UID instead of the username.

Example:

1001 file.txt

Because the system cannot find the username anymore.

5 Problem with UID on Multiple Systems

Unlike Windows SIDs, Linux UIDs are not globally unique.

Example:

System A:

user1 → UID 1001

System B:

hacker → UID 1001

If these systems share files:

hacker may access files from user1.

This creates security problems in network file sharing.

The Superuser (Root)

Linux has a two-level security model:

Type	Access
Normal User	Limited access
Superuser	Full control

The superuser can:

read all files

modify everything

control processes

control devices

Root User

The superuser is usually called:

root

But the name does not matter.

What matters is the UID.

Superuser UID = 0

Any account with:

UID = 0

is a superuser.

Example:

root UID=0
admin UID=0
system UID=0

All of them have full system control.

Security Risk of UID 0

Attackers often try to:

create new UID 0 accounts

modify an existing account to UID 0

Because that gives complete system access.

Example malicious entry:

hacker:x:0:0:/root:/bin/bash

Now the attacker has root privileges.

Logging Problem

If multiple users have UID 0, logs cannot distinguish them.

Example log:

UID 0 deleted file

But which user?

root

admin

hacker

Impossible to know.

This makes auditing very difficult.

Special UID 0 Accounts

Sometimes systems contain extra UID 0 accounts for applications.

Example:

database service

backup tools

system services

These accounts are usually:

locked

cannot login

Attackers often target these accounts.

Therefore admins must audit UID 0 accounts regularly.

Simple Summary

User Accounts

Username/password are case sensitive

Linux actually uses UID/GID numbers

Important Files

/etc/passwd → user information

/etc/shadow → encrypted passwords

Superuser

UID 0 = root privileges

Any account with UID 0 has full control

Security Risks

Multiple UID 0 accounts

Attackers creating new UID 0 users

File sharing problems due to duplicate UID

Privilege elevation means getting root (superuser) permissions from a normal user account in Linux. There are three main methods:

Login as Root

Directly log in using the root account (UID 0).

Called anonymous login because actions cannot be tied to a specific user.

Dangerous and not recommended due to lack of accountability.

Should only be used in emergencies and preferably only from the console, not over the network.

su Command

Log in as a normal user first.

Use su to switch to another user, usually root.

Requires the target user's password (often root’s password).

Provides better accountability because logs show which user switched to root.

sudo Command

Log in as a normal user.

Use sudo to run specific commands with root privileges.

User must be listed in /etc/sudoers.

Advantages:

Only requires the user’s own password

Allows granular control (specific commands allowed)

Better logging and auditing

✅ Best practice: Use sudo instead of direct root login or su because it is more secure and provides better control and logging.

Summary: Sudo and Sudoers

In Linux, direct login as root is discouraged because it provides no accountability (you cannot know which user performed actions).

Administrators should log in with their own accounts and then elevate privileges when needed.

Traditionally, admins used su (switch user) to become root.

Problem: su gives full root access, with no fine control over what commands a user can run.

sudo solves this problem by allowing granular control:

Users can run only specific commands with root privileges.

Each user uses their own password, improving security and accountability.

The sudoers file (/etc/sudoers) defines:

Who can use sudo

Which commands they can run

On which systems

As which users

Example:

root, admin group, and sudo group → full privileges.

user sans → can only run useradd, mkdir, and rmdir.

sans must enter a password for some commands, but not for others.

✅ Key idea:
sudo = controlled root access, and sudoers = rules that define those permissions.

System accounts are special accounts used by applications or services, not by normal users.
Examples: nobody (NFS), oracle (Oracle database), apache or www (web server).

These accounts usually have low UID numbers to distinguish them from regular users.

Red Hat: UID < 500

Debian: UID < 1000

Solaris: UID 0–99

System accounts are typically disabled by default with invalid passwords in /etc/shadow to prevent login.

Attackers may try to add valid passwords to these accounts as a backdoor, so administrators should:

Set invalid shells in /etc/passwd

Regularly audit /etc/passwd and /etc/shadow.

It is usually better to disable accounts instead of deleting them, because:

Files in UNIX are owned by UID numbers.

If an account is deleted, its UID might be reused, giving a new user ownership of important system files.

The same idea applies when employees leave: accounts are often blocked instead of deleted to avoid UID conflicts and maintain file ownership history.

✅ Key idea:
System accounts run services and are usually disabled for security, and disabling is often safer than deleting to avoid UID ownership issues.

password file is separated by a colon (:), as follows:
• 1st: User ID
• 2nd:Location for the encrypted password (normally an x to indicate the system is shadowed)
• 3rd:User identification number
• 4th:User's primary group
• 5th:GECOS field (remarks)
• 6th:User's home directory
• 7th:What happens upon login (normally this will be a process

Shadow File
sans:$6$0.Ba4RRKwEGyMu.c$UBwvPHtjX1uYNNMPM7HougGTay0VpTDqkHKbj2EuTu38.aU4
XVPtNYz8fwUcF/iXvei8lTg7eFYALtP37cOoo/:18681:0:99999:7:::
Username: Name matching the entry in the /etc/passwd file
Password: Encrypted password along with the salt and type of encryption
Last: Number of days since the last password change based on 1 Jan 1970
Minimum: Number of days before the password can be changed again
Maximum: Number of days after which the password must be changed
Warn: Number of days before the password must be changed, so the user gets a warning message
Expire: Number of days before the account becomes disabled after its password expired
Disable: Number of days before the account becomes disabled
Reserved: Not used at this time

PAM (Pluggable Authentication Modules) is a Linux system library that manages authentication (how users log in and prove their identity).

Instead of every program handling login security itself, PAM provides a central system that applications can use for authentication. This makes Linux more secure and easier to manage.

1 What PAM does

PAM helps control:

User login

Password checking

Access permissions

Session management

Many services use PAM, such as:

SSH

login

sudo

graphical logins

2 The four PAM management groups

PAM divides authentication tasks into four groups:

auth (Authentication)

Verifies who the user is.

Example: checking username and password.

password

Handles password changes and updates.

session

Manages actions when a user logs in or logs out.

Example: creating a home directory or logging activity.

account

Checks account rules, not identity.

Example: account expiration or login time restrictions.

3 Where PAM configuration is stored

All PAM configuration files are in:

/etc/pam.d/

Each file controls one service.

Examples:

/etc/pam.d/login

/etc/pam.d/sshd

/etc/pam.d/sudo

4 PAM configuration format

Each line in a PAM configuration file usually looks like this:

Type Control Module-path Module-arguments

Meaning:

Type → which PAM group (auth, account, password, session)

Control → what to do if the authentication fails

Module-path → the module used for authentication

Module-arguments → extra options for the module

Triple Threat: ls, ps, netstat (Easy Explanation)

In cybersecurity, many attacks follow a common pattern:

The attacker uploads a malicious file.

The file runs as a background process.

The process connects to an external server (called C2 / Command & Control).

Security analysts can detect this behavior using three simple Linux tools: ls, ps, and netstat.

1 ls – Check Files

ls lists the files in a directory.

Security use:

See which files exist on the system

Look for new or suspicious executables

Example:

ls -la

Admins create a baseline (a normal list of files).
Later they check again to detect new or modified files that might be malicious.

2 ps – Check Running Processes

ps shows programs currently running on the system.

Security use:

Identify unknown or suspicious processes

See if a malicious file is running in the background

Example:

ps aux

Admins compare the current processes with the normal baseline.

3 netstat – Check Network Connections

netstat shows network connections and ports.

Security use:

Detect unexpected outbound connections

Identify malware communicating with a C2 (Command & Control) server

Example:

netstat -tulnp

4 How they work together

Security analysts use these three tools together:

Tool	What it checks
ls	Files on the system
ps	Running processes
netstat	Network connections

This helps detect:

New malicious files

Suspicious running programs

Unauthorized external connections

The tail command in Linux is used to display the last part of a file, often used to read log files.

It is very useful in security monitoring because logs are constantly updated.

1 tail -f (Follow file updates)

The -f option allows you to continuously monitor a file.
When new lines are added, they appear automatically on the screen.

Example:

tail -f /var/log/messages

This is useful for real-time monitoring of system logs.

You can also control the refresh time with -s (sleep):

tail -f -s 5 /var/log/messages

Meaning:

Check the file every 5 seconds for updates.

2 tail -c (Show bytes)

Normally, tail shows the last lines of a file.
But with -c, you can show the last number of bytes (characters).

Example:

tail -c 500 /var/log/messages

This displays the last 500 bytes of the log file.

3 Security example

Security analysts often monitor firewall logs like:

/var/log/messages

Using tail -f, they can watch attacks in real time, such as:

Port scanning

Unauthorized connections

Suspicious activity

Example from logs:

Host 192.168.150.133 scanned ports on 192.168.150.134

It tried to check port 22 (SSH).

Finding the Needle in the Haystack: | (pipe) and grep — Easy Explanation

In cybersecurity, every attack leaves traces (clues) in system logs.
The challenge is that logs contain huge amounts of data, making it hard to manually find suspicious activity.

To solve this problem, Linux provides powerful tools like pipes (|) and grep to filter and analyze data quickly.

1 Pipe | (connecting commands)

The pipe (|) sends the output of one command as input to another command.

This helps automate analysis and process large amounts of data quickly.

Example:

command1 | command2

Meaning:

command1 produces output

command2 processes that output

Example with logs:

tail /var/log/auth.log | grep ssh

This:

Shows the latest log lines

Filters only lines containing ssh

2 grep (search for patterns)

grep is used to search for specific words or patterns in files or command output.

Example:

grep ssh /var/log/auth.log

This shows only lines containing ssh.

Security analysts use grep to find:

Failed logins

Suspicious IP addresses

Attack attempts

Error messages

3 Example: Detecting an SSH attack

Earlier logs showed an attacker scanning ports and finding port 22 (SSH) open.

To check if the attacker tried to log in, analysts can run:

tail -f /var/log/auth.log | grep sshd

This command:

Monitors the log in real time (tail -f)

Filters SSH daemon messages (grep sshd)

From the logs, we might see:

Two failed password attempts

Then a successful login using the user sans

This indicates a possible password compromise.

Systemd Overview

Systemd is a software suite in Linux that replaces the traditional init daemon. It is responsible for starting and managing processes and services, providing a unified way to manage Linux systems across distributions.

Key Features of Systemd

Full System and Service Management

Manages system services, including starting, stopping, and monitoring.

Supports parallel processing, so multiple services can start simultaneously.

Monitors services after boot and can restart them if they fail.

Supports device hot-plugging.

Software Platform

Provides a platform to maintain, create, and manage services.

You can add new services or modify existing ones without rebooting the system.

Interface to Kernel

Acts as a controlled bridge between applications and the kernel.

Limits direct kernel access while still providing required functionality.

Reduces risk of system compromise while allowing apps to function.

Logging with journald

Systemd uses journald, a binary log system.

Logs are not traditional text syslogs, which may require special tools to analyze.

This is important for security monitoring, troubleshooting, and auditing.

Systemd Commands

Some common commands to manage services:

Command	Purpose
systemctl start name.service	Start a service
systemctl stop name.service	Stop a service
systemctl restart name.service	Restart a service
systemctl reload name.service	Reload configuration
systemctl status name.service	Check service status
systemctl is-active name.service	Check if service is active
systemctl list-units --type service --all	List all services

Architecture and Components

Systemd is more than just an init daemon; it is a suite of software including:

systemd init daemon – starts and manages services.

journald – handles logging.

logind – manages user logins.

networkd – handles network services.

Other low-level components to manage timers, cgroups, and sessions.

Security Considerations

Reduces attack surface by controlling which services and processes run.

Provides integrated service management to ensure system stability.

Kernel interface is controlled, limiting the potential for exploits.

The following are some of the common options for APT:

• apt-get update: Updates the system database of software updates that are available.
• apt-get install package-name: Installs the specified software with all associated dependencies.
• apt-get remove package-name: Removes the specified software but does not remove dependencies.
• apt-get update: Updates the system database of software updates that are available.
• apt-get upgrade: Upgrades all the software with the available updates.

2. SELinux Adds Stronger Security (MAC)

SELinux = Security-Enhanced Linux

It adds a stronger model called Mandatory Access Control (MAC).

Instead of users deciding access, the system enforces strict policies.

So even if a program has Linux permissions, SELinux can still block it.

Think of it like this:

Linux permissions = door lock

SELinux = security guard checking who enters

3. How SELinux Works (Simple Idea)

SELinux uses 3 main concepts:

1 Users

Who is using the system.

Example:

admin
user1
apache
2 Roles

What the user is allowed to do.

Example:

user role
admin role
system role
3 Domains

What programs/processes are allowed to access.

Example:

httpd_t → Apache server
ssh_t → SSH service

Each program runs inside a domain with limited permissions.

4 File Types (Labels)

Every file has a security label.

Example:

/var/www/index.html → httpd_sys_content_t

This tells SELinux:

Only the Apache process can read it.

4. Simple Example

Imagine a web server (Apache).

Without SELinux:

If Apache is hacked, the attacker could read any file Apache user can access.

With SELinux:

Apache is only allowed to read web files.

It cannot read system files like /etc/shadow.

So the attack stops.

5. Key Idea: Least Privilege

SELinux follows the rule:

Give programs only the permissions they absolutely need.

Example:

Program	Allowed
Apache	Read web files
SSH	Handle login
Database	Access database files

They cannot access each other's data.

6. Labels in SELinux

SELinux attaches labels to files and processes.

Example format:

user:role:type:level

Example:

system_u:object_r:httpd_sys_content_t:s0

Meaning:

system user

object role

Apache web content type

security level

7. MLS / MCS (Security Levels)

SELinux can also use security levels like military systems.

Example:

Top Secret
Secret
Confidential
Public

Format:

sensitivity:categories

Example:

s0:c0,c1,c2

But most Linux systems do not use full MLS.

8. Important Point

SELinux does NOT replace Linux security.

It adds another layer.

So Linux uses:

1 DAC (normal permissions)
2 MAC (SELinux rules)

Together they make the system much more secure

1. What is AppArmor?

AppArmor is a Linux security module that limits what programs can do.

It protects the system by restricting applications so they cannot access files or resources they shouldn't.

Think of it like this:

A program normally can access many parts of the system.

AppArmor creates rules to limit that program.

Example:
A web server should only access web files, not system passwords.

2. Simple Idea

Without AppArmor:

Program → Can access many system resources

With AppArmor:

Program → Only allowed to access specific files and actions

So if a program is hacked, the attacker cannot control the whole system.

3. How AppArmor Works

AppArmor uses profiles.

A profile is a file that defines:

what files a program can access

what actions it can perform

what system resources it can use

Example idea:

Apache Web Server
Allowed:
/var/www/*

read files
Blocked:
/etc/shadow
system files
4. Behavior-Based Protection

AppArmor can learn how applications normally behave.

Then it creates rules based on that behavior.

This helps:

detect unusual actions

block suspicious activity

stop zero-day attacks

It uses:

static analysis (checking code)

learning-based tools

5. AppArmor Profiles

Each program has its own security profile.

Example profile path:

/etc/apparmor.d/

Example program profile:

/etc/apparmor.d/usr.sbin.nscd

This profile controls the program:

/usr/sbin/nscd

which is a Name Service Cache Daemon (speeds up user lookup operations).

6. Profile Components (Simple)
1 Include statements

These lines include common rules.

Example:

#include <abstractions/base>

They provide common permissions like:

authentication

network access

system functions

2 File Access Rules

Lines starting with / define files the program can access.

Example:

/etc/passwd r

Meaning:

program can READ /etc/passwd

3 Network Permissions

Some rules allow the program to open network ports.

Example:

network inet

This allows network communication.

4 Variables

Lines starting with @ are variables used for paths.

Example:

@{HOME}

This represents:

/home/*

Variables make profiles portable and easier to manage.

7. File Permission Modes

AppArmor profiles use letters to define access permissions.

Permission	Meaning
r	Read file
w	Write file
a	Append to file
k	Lock file
l	Create hard link

Example:

/var/log/app.log rw

Meaning:

Program can read and write this file

8. Why AppArmor is Useful

AppArmor helps:

✔ Stop attackers if a program is hacked
✔ Prevent malicious software from spreading
✔ Limit damage inside the system

Example attack:

Web server exploited

Without AppArmor:

Attacker reads system files

With AppArmor:

Access denied

9. AppArmor vs SELinux (Simple Difference)

Feature	AppArmor	SELinux
Difficulty	Easy	Complex
Control method	File paths	Security labels
Configuration	Profiles	Policies
Learning mode	Yes	Limited

Many Linux distributions use:

Ubuntu → AppArmor

RedHat / CentOS → SELinux

1. What is Sysctl?

Sysctl is a Linux tool used to change kernel settings while the system is running.

The kernel is the core of Linux, and sysctl lets administrators adjust how the kernel behaves.

You can use it to control things like:

networking

memory

security protections

system behavior

Main configuration file:

/etc/sysctl.conf

Changes in this file stay after reboot.

2. Two Ways to Use Sysctl

1 Temporary changes (runtime)

You can change a kernel setting immediately using:

sysctl -w variable=value

Example:

sysctl -w net.ipv4.tcp_syncookies=1

This change disappears after reboot.

2 Permanent changes

Edit the configuration file:

/etc/sysctl.conf

Example:

net.ipv4.tcp_syncookies = 1

Then apply changes:

sysctl -p

Now the change remains after reboot.

3. Viewing All Kernel Variables

To see all sysctl settings in your system:

sysctl -a

This shows hundreds of kernel variables.

Many of them are also stored in:

/proc/sys/

Example:

/proc/sys/net/ipv4/

These files represent kernel settings.

4. What is Sysctl Hardening?

Hardening means making a system more secure against attacks.

Sysctl hardening changes kernel parameters to:

block network attacks

log suspicious packets

prevent spoofing

protect against flooding attacks

5. Example Security Settings
1 Log suspicious packets
net.ipv4.conf.all.log_martians = 1

This logs impossible or fake IP packets.

Example attack:

Packet from impossible address

Linux logs it for investigation.

2 Disable source routing
net.ipv4.conf.all.accept_source_route = 0

Source routing lets attackers control packet paths.

Turning it off improves security.

3 Disable redirect packets
net.ipv4.conf.default.accept_redirects = 0

Redirect packets can be used in Man-in-the-Middle attacks.

Disabling them prevents attackers from changing network routes.

4 Disable secure redirects
net.ipv4.conf.default.secure_redirects = 0

Prevents attackers from sending fake routing messages.

5 Protect against SYN Flood attacks
net.ipv4.tcp_syncookies = 1

A SYN flood is a Denial-of-Service attack.

Attackers send many connection requests to overload a server.

Syncookies help protect against this attack.

6 Control connection retries
net.ipv4.tcp_synack_retries = 5

Limits how many times the system retries a connection.

This helps reduce resource exhaustion attacks.

6. Memory Exploit Protections

Sysctl also supports security protections like:

1 ASLR

Address Space Layout Randomization

ASLR randomly changes memory locations.

This makes buffer overflow exploits much harder.

Example:

Without ASLR:

Attackers know where memory addresses are

With ASLR:

Memory addresses change every run

Attackers cannot predict them.

2 Exec Shield

Exec Shield protects against buffer overflow attacks by preventing code execution in unsafe memory regions.

It helps stop:

stack overflow attacks

memory injection attacks

ASLR is a security technique in Linux that randomizes memory addresses used by programs.

Normally, when a program runs, its components are loaded into memory:

program code

libraries

stack

heap

Without protection, these locations can be predictable.

ASLR randomizes these memory locations every time the program runs.

What is a Kernel Module?

A kernel module is a piece of code that can be added to the Linux kernel to give it new functionality.

Examples:

drivers (USB, Wi-Fi, Bluetooth)

filesystems

networking features

Instead of putting everything in the kernel, Linux loads modules only when needed.

Example modules:

usb_storage
bluetooth
vfat
2. What is Modprobe?

modprobe is a Linux command used to:

load kernel modules

remove kernel modules

manage dependencies between modules

Examples:

Load a module:

modprobe usb_storage

Remove a module:

modprobe -r usb_storage

1. Kernel Modules in Linux

Linux kernel features can come from two types of modules:

1 Built-in modules

These modules are compiled directly into the kernel.

Always available

Loaded when the system starts

Cannot be removed or added later

Example idea:

```
Kernel
 ├─ filesystem support
 ├─ networking
 └─ drivers
```

2 Loadable Kernel Modules (LKM)

These modules can be loaded or removed while the system is running.

Example:

modprobe usb_storage

This loads the USB storage driver without rebooting.

✔ Advantage: flexibility
❌ Disadvantage: security risk

2. Why Dynamic Loading is a Security Risk

If an attacker gets root privileges, they can load a malicious kernel module.

That module could:

install a kernel rootkit

hide processes

hide files

intercept system calls

spy on network traffic

Because kernel modules run in kernel space, they have full control of the system.

Example attack idea:

Attacker → root access
↓
Load malicious kernel module
↓
Rootkit hides attacker activity
3. Dynamic Loading After Boot

Normally Linux allows modules to be loaded any time after boot.

Example:

modprobe module_name

This flexibility is useful but dangerous for high-security systems.

4. Security Hardening Idea

After the system finishes booting, you can disable loading new kernel modules.

This prevents attackers from inserting malicious kernel code.

What is SSH?

SSH (Secure Shell) is a protocol used to remotely log in to a Linux system securely.

Example:

ssh user@192.168.1.10

System administrators use SSH to:

manage servers

transfer files

run commands remotely

Because SSH gives remote control of the system, it is a major target for attackers.

SSH Hardening (Making SSH More Secure)

Hardening means configuring SSH to reduce security risks.

The slide lists 6 important SSH hardening techniques.

1. Use TCP Wrappers (Allow Specific Hosts)

You can restrict which IP addresses are allowed to connect to SSH.

Files used:

/etc/hosts.allow
/etc/hosts.deny

Example:

Allow only one IP:

sshd: 192.168.1.100

Block everyone else:

sshd: ALL

This prevents unknown computers from connecting to SSH.

2. Disable Root Login

By default, attackers try to log in as:

root

Because root has full control of the system.

So it is safer to disable root SSH login.

Edit the SSH configuration file:

/etc/ssh/sshd_config

Change:

PermitRootLogin no

Now attackers cannot log in directly as root.

Admins must log in as a normal user and then use:

sudo
3. Set Idle Timeout Interval

If someone connects via SSH but leaves the session open, it can be hijacked.

You can force SSH to disconnect inactive sessions.

In /etc/ssh/sshd_config:

ClientAliveInterval 300
ClientAliveCountMax 0

Meaning:

after 5 minutes of inactivity

the SSH session automatically closes

4. Disable Empty Passwords

Sometimes accounts may have no password, which is extremely dangerous.

Disable this by setting:

PermitEmptyPasswords no

Now users must have passwords.

5. Set a Custom SSH Warning Banner

You can display a security warning message before login.

Example:

WARNING: Authorized users only.
All activity is monitored and logged.

Configure in /etc/ssh/sshd_config:

Banner /etc/issue.net

Then edit the banner file:

/etc/issue.net

Purpose:

warns attackers

supports legal monitoring policies

6. Block SSH Brute Force Attacks

Attackers often try thousands of passwords to break SSH.

This is called:

Brute Force Attack

Example attack:

root:123456
root:password
root:admin
root:qwerty

To stop this you can use tools like:

Fail2Ban

DenyHosts

Example protection:

Too many failed logins → IP address blocked

What is Multifactor Authentication (MFA)?

Multifactor Authentication (MFA) means using more than one method to verify a user.

Instead of logging in with only a password, the system asks for two or more authentication factors.

The idea is based on:

Factor Type	Meaning	Example
Something you know	Knowledge	Password / PIN
Something you have	Possession	Phone / token
Something you are	Biometrics	Fingerprint / Face

For SSH, MFA usually uses:

Password + Authentication code

or

SSH Key + Authentication code

2. Why MFA is Important for SSH

SSH is used to remotely control servers.

If attackers steal or guess a password, they could take full control of the system.

Example attack:

Attacker guesses password → SSH login → Server compromised

With MFA:

Password → Correct
Authentication code → Required

Without the second factor, the attacker cannot log in.

3. Common MFA Methods for SSH

1 SSH Keys

SSH keys replace passwords with cryptographic keys.

Two parts:

Private key → stored on your computer

Public key → stored on the server

Login example:

ssh user@server

The server verifies the key instead of asking for a password.

✔ More secure than passwords.

2 One-Time Password (OTP) Apps

These apps generate temporary login codes that change every 30 seconds.

Example code:

843921

Even if someone steals the code, it expires quickly.

3 Popular MFA Applications

Several tools can generate these authentication codes.

Google Authenticator

Very popular

Open-source

Works on Android and iPhone

Generates 6-digit OTP codes

FreeOTP

Open-source alternative

Developed by Red Hat

Works similarly to Google Authenticator

Authy

Cloud backup for tokens

Multi-device support

Easy recovery

Duo

Enterprise security solution

Used by companies

Supports push notifications

SSH/TLS Key Management is the process of securely managing cryptographic keys used for authentication.

Problems:

private keys stored locally

difficult to track access

risk of stolen keys

Solutions:

centralized key management

tools like KeyBox

automation tools like Puppet

adding MFA authentication

This improves security, monitoring, and access control.

Default Linux Installation Is Not Fully Secure

When you install Linux for the first time, the system is not perfectly secure.

This is because Linux distributions try to be usable for many situations (desktop, servers, labs, etc.).

So they often include:

extra services

default settings

open features

These can create security risks.

2. Problem 1 — Extra Services Installed

Many Linux installations include additional services that you may not need.

Examples:

FTP server

web server

printing services

database services

Example idea:

Linux installed
↓
Many services running
↓
More attack surfaces

If attackers find a vulnerability in one of these services, they could compromise the system.

3. Problem 2 — Default Configurations May Be Weak

Default configurations are designed for convenience, not maximum security.

Examples of weak defaults:

SSH root login enabled

weak password policies

unnecessary network services

open ports

Attackers often know these default settings and try to exploit them.

4. Problem 3 — Users May Not Know How to Secure Linux

Not every Linux user is a security expert.

Many users:

install Linux

run services

leave default settings

This means systems may remain misconfigured and vulnerable.

5. Problem 4 — Manual Security Configuration Takes Time

Hardening a Linux system manually requires many steps.

Examples:

disable services

configure firewall

secure SSH

configure logging

enable kernel protections

manage permissions

Example process:

Install Linux
↓
Configure firewall
↓
Configure SSH
↓
Disable services
↓
Configure logging

Doing this manually on many servers can be very slow.

6. Solution — Hardening Scripts

A hardening script is a program that automatically secures the system.

Purpose:

reduce vulnerabilities

apply security settings

disable unnecessary services

configure protections

Example idea:

Run hardening script
↓
System automatically secured

7. What Hardening Scripts Do

Typical tasks include:

disabling unnecessary services

configuring SSH security

enabling firewall rules

enforcing password policies

enabling logging

applying kernel protections

Example command:

```
./linux_hardening.sh
```

8. Warning About Hardening Scripts

Hardening scripts can sometimes cause problems.

Too much security can break system functionality.

Example:

Script disables service
↓
Application stops working

So administrators must understand what the script changes.

9. Best Practice

Security professionals follow this workflow:

1. Test hardening script in development
2. Verify system still works
3. Apply changes in production

This prevents service outages.

10. Change Control Process

In professional environments, system changes must follow a change control process.

This means:

documenting the change

testing it

approving it

then applying it to production systems

This ensures stability and security.

What are Hardening Scripts?

They are automated scripts that help make a computer system more secure by applying security settings. Think of them like a “security recipe” you run on many computers at once.

Pros (Why They’re Good)

Scalable: You can secure lots of systems quickly without doing each one manually.

Consistent Security: Everyone gets the same security settings, reducing mistakes.

Automated: Saves time and effort for system administrators.

Auditing Help: You can use them to prove that systems follow security rules.

Cons (Risks / Cautions)

Blind Changes: If the person running the script doesn’t understand it, it can break important system services.

No Clear Metrics: Hard to check if the security changes actually make the system safer.

Learning Gap: The admin may not learn or understand security deeply—they’re just running a script.

Not One-Size-Fits-All: Different systems need different settings; a generic script may not fit all cases.

What is Lynis?

Lynis is a tool that checks the health and security of your Linux/UNIX system. Think of it as a “security doctor” for your computer—it scans and tells you what’s wrong or risky. It’s mostly used for auditing systems and helping meet compliance rules like PCI DSS, HIPAA, SOX, and others.

What Lynis Can Check

Lynis scans for a wide range of things, including:

Authentication methods: Are users logging in securely?

Expired SSL certificates: Are website or server certificates still valid?

Outdated software: Are programs running old, vulnerable versions?

User accounts without passwords: Are there risky accounts?

Incorrect file permissions: Are files too open or insecure?

Firewall settings: Is your firewall configured correctly?

Pros of Lynis

Comprehensive auditing: Checks lots of security and system settings.

Compliance-ready: Helps show that your system follows regulations like SOX or PCI.

No installation required: You can run it directly on your system.

Enterprise support: With the Enterprise edition, you can scan many systems and see all results in one place.

Limitations

Single system focus by default: Without the enterprise version, it’s mainly useful for auditing one system at a time.

Not automatic hardening: Lynis reports issues but doesn’t automatically fix them. You need to make changes manually.

What is OpenSCAP?

OpenSCAP is a set of open-source tools created by Red Hat to help organizations automate security checks and compliance auditing for Linux systems. It uses a standard called SCAP (Security Content Automation Protocol) to make sure systems meet security policies and are free from known vulnerabilities.

What is InSpec?

InSpec is an open-source framework that lets you test and enforce security and compliance rules on your servers, whether they're on-premise or in the cloud. Think of it as a "unit test for your infrastructure"—it checks if your systems are configured the way they should be.

Key Log Files

UNIX systems use multiple key log files, several of which cannot be viewed by using any kind of text parsing
tool directly and must be used with their own specific log viewing utility.

• utmp:Gives a complete picture of users' logins including which terminals were used, logouts, system
events, current status of the system, and system boot time.
• wtmp:Gives historical data of utmp.
• btmp:Records only failed login attempts.
• dmesg:Is a display or driver message. It is used to examine or control the kernel ring buffer.
• messages: Contains global system messages, including the messages that are logged during system
startup. Several things are logged in /var/log/messages, including mail, cron, daemon, kern, and auth.
• maillog: Contains the messages from the mail server running on the system, such as sendmail.
• secure: Contains all security-related messages on the system. This includes authentication failures,
possible break-in attempts, SSH logins, failed passwords, sshd logouts, and invalid user accounts.

Key Log Files

Linux has three main log files for user login information:

utmp – Tracks currently logged-in users.

wtmp – Keeps historical login/logout records.

btmp – Logs failed login attempts.

How to Read Them

These files are not plain text, so you can't just open them with cat.

Use special commands:

who → shows who is logged in (reads utmp).

last → shows login history (reads wtmp).

lastb → shows failed login attempts (reads btmp).

utmpdump → converts utmp/wtmp/btmp into a readable table.

utmpdump Output Columns

When you use utmpdump to read a file, it shows a table like this:

Type of record (ut_type)

PID of login process (ut_pid)

Terminal name (ut_id)

Username (ut_user)

Device or tty (ut_line)

Remote host or kernel host (ut_host)

Internet address of remote host (ut_addr_v6)

Time of entry (ut_time or ut_tv.tv_sec)

What is dmesg?

dmesg is both a log file and a command used to read that log.

It contains messages from the kernel about the system boot process and hardware drivers.

Think of it as the system's "startup diary", recording everything the kernel and drivers do from power-on until the OS is fully running.

Why dmesg is Useful

Troubleshooting boot issues: If your system behaves strangely when starting, dmesg can show the cause.

Hardware monitoring: Shows driver messages when devices like USBs or disks are plugged in.

Kernel messages: Reports kernel module loading, driver parameters, and errors.

Key Points

dmesg shows kernel ring buffer, which is a temporary memory of kernel messages.

Messages can include:

Drivers loading successfully or failing

Errors in hardware detection

Diagnostic messages during boot or when new devices are connected

Example Usage

dmesg | less → Scroll through kernel messages.

dmesg | grep usb → See messages related to USB devices.

Helps identify issues during startup or with hardware

Key Log Files: messages
Dmesg and messages often are thought to contain the same logging information. However, they do serve

different purposes: dmesg captures only the kernel's messages of any log level, whereas the messages log
stores valuable, non-debug, and noncritical messages. The messages log should be considered the general
system activity log, which is a fantastic resource for troubleshooting and anomaly hunting.
The messages log file is generally located in /var/log/messages

Syslog Overview

Purpose:
Syslog is the standard system for logging messages on Linux and UNIX-like systems. It allows almost any part of the system—applications, drivers, or daemons—to record events.

Key Component:
The main process is the system logging daemon (syslogd), which is typically started during system boot via startup scripts (like rc scripts in run level 1).

Flexibility:

Supports log entries from various sources: system components, applications, drivers, and other daemons.

Offers a command-line interface so scripts or users can generate log entries manually.

Message Handling:

Each log entry is associated with a facility code that identifies the type of program generating the message.

Different facilities can be treated differently—for example, system logs might go to /var/log/messages, while authentication logs go to /var/log/auth.log.

History:

Developed in the 1980s by Eric Allman as part of the Sendmail project.

Quickly became the standard logging solution on UNIX-like systems.

Implementations exist for other operating systems and network devices like routers.

Syslog Security — Easy Explanation

1 What is Syslog Security?

Syslog is a system that sends and stores log messages (events) from devices like servers, routers, firewalls, and applications.

These logs can include things like:

Login attempts

System errors

Network activity

Security alerts

Because logs may contain important security information, we must protect them.

Two main security concerns are:

Sender Authentication

This means verifying who sent the log message.

Without authentication:

An attacker could send fake log messages

This could hide real attacks or confuse investigators

Example:

A hacker sends fake logs saying “Everything is normal” while attacking the system.

So systems should verify:

The real device sent the log.

Message Confidentiality

This means protecting log messages from being read by others.

Without confidentiality:

Anyone on the network could intercept the logs

Logs might contain IP addresses, usernames, errors, or system info

Solution:

Encrypt the logs when sending them (for example using TLS).

Syslog Standards

There are two main syslog standards.

1 BSD Syslog (Old / Legacy)

This is the older and more common standard.

Problems:

Message size limit
Maximum 1024 bytes per packet
→ Long logs may be cut off (truncated)

⏱ Low time precision
Time recorded only to the second

Example:

10:15:32 login attempt failed

But many attacks happen within milliseconds, so this may not be accurate enough.

2 IETF Syslog (New Standard)

This is the modern and better version.

Advantages:

✅ No message size limit

✅ More precise timestamps (milliseconds)

✅ Transport independent (can work with different protocols)

✅ Better security support

Example timestamp:

10:15:32.456

This helps detect very fast attacks.

Real-World Situation

Even though IETF Syslog is better, many devices (routers, firewalls, appliances) still use:

➡ BSD Syslog

because it is older and widely supported.

Syslog-NG is an improved version of Syslog that provides better filtering, secure log transmission (TLS), TCP reliability, and advanced log management

What is auditd?

auditd (Audit Daemon) is a Linux security monitoring system that records detailed system activity.

It was developed by Red Hat and works directly with the Linux kernel.

That means it can monitor almost everything happening on the system.

Example activities auditd can track:

file access

directory access

network connections

system calls

configuration changes

SELinux events

Why auditd is Important

auditd is mainly used for:

Security monitoring
Forensics (investigating attacks)
System auditing and compliance

Example:

If someone modifies a critical file like:

/etc/ssh/sshd_config

auditd can record:

who changed it

when it happened

what program was used

from which directory

What auditd Can Monitor

auditd can log:

1 File Access Logging

Tracks when files are:

read

written

modified

executed

Example monitored file:

/etc/passwd

2 Directory Access Logging

Tracks when someone accesses or modifies a directory.

Example:

/etc/

3 Network Activity

auditd can monitor network-related system calls.

4 SELinux Logs

auditd is also used to record SELinux security events.

auditd Configuration

auditd rules can be controlled by:

1 Rule File
/etc/audit/rules.d/audit.rules

These rules are persistent (remain after reboot).

2 auditctl Command

Used to create temporary rules.

Example command:

```
sudo auditctl -w /etc/ssh/sshd_config -p rwxa -k SANSsshchange
```

Meaning:

Part	Meaning
-w	watch file
/etc/ssh/sshd_config	file to monitor
-p rwxa	monitor read, write, execute, attribute changes
-k SANSsshchange	label for this rule

So any access to this file will generate a log.

Important auditctl Commands
List Rules
auditctl -l

Shows all active audit rules.

Show auditd Status
auditctl -s

Displays audit daemon status.

Delete All Rules
auditctl -D

Removes all audit rules.

Lock Configuration

auditctl -e 2

Locks audit rules so no one can modify them until reboot.

This is useful for high-security systems.

Tools Used with auditd

auditd provides several tools to analyze logs.

1 ausearch

Search through audit logs.

Example:

sudo ausearch -f /etc/ssh/sshd_config -i

Meaning:

search logs related to that file

-i converts numbers into readable text

This helps find who accessed the file.

2 aureport

Creates human-readable reports.

Example:

sudo aureport -f -i --summary

Shows:

number of events per file

which files were accessed most

Example filtered output:

sudo aureport -f -i | grep sshd_config

This shows:

date

user

program used

Example result:

User SANS accessed sshd_config using nano
3 autrace

Used to trace a program's system calls.

Example:

autrace ls

This shows what system calls the program uses.

This helps administrators decide what to monitor with audit rules.

Where audit Logs Are Stored

Usually here:

/var/log/audit/audit.log

These logs are very detailed and often very long.

Important Note

auditd does NOT use syslog.

It has its own logging system.

That means:

log collection

remote logging

log management

must be done using auditd tools.

Example Security Scenario

Administrator wants to monitor SSH configuration changes.

Step 1 — Create rule:

sudo auditctl -w /etc/ssh/sshd_config -p rwxa -k sshchange

Step 2 — Someone edits the file:

nano /etc/ssh/sshd_config

Step 3 — Search logs:

ausearch -f /etc/ssh/sshd_config -i

After auditd collects security logs, administrators need tools to search, analyze, and understand those logs.

The three main auditd tools are:

1 ausearch → search audit logs
2 aureport → generate reports from logs
3 autrace → trace system calls of programs

1 ausearch (Search Audit Logs)

ausearch is used to search inside audit logs.

Audit logs are very detailed and large, so this tool helps find specific events.

Example location of audit logs:

/var/log/audit/audit.log
Search by Rule Key
ausearch -k SANSSSHchange

Meaning:

Search for logs generated by the rule with key SANSSSHchange.

Search Login Events
ausearch -m LOGIN --start today -i

Meaning:

Option Meaning
-m LOGIN search login events
--start today only today's logs
-i interpret numeric values as readable text
Search by Event ID
ausearch -a 7618

Meaning:

Show the audit event with ID 7618.

Search by File
ausearch -f /etc/ssh/sshd_config

Meaning:

Find all events involving that file.

Example result:

User opened sshd_config using nano
2 aureport (Create Audit Reports)

Because audit logs are very long and complex, aureport converts them into readable summaries.

Command Execution Summary
aureport -x --summary

Shows:

programs executed

number of times they ran

Example output idea:

nano 5 executions
ssh 10 executions
sudo 3 executions
Failed Login Attempts
aureport --failed

Shows:

failed authentication attempts

login failures

Useful for detecting brute-force attacks.

File Access Report
aureport -f -i

Shows:

which files were accessed

which program accessed them

when it happened

3 autrace (Trace Program Activity)

autrace tracks the system calls a program makes.

System calls are the requests programs send to the Linux kernel.

Example:

autrace /usr/bin/find

This shows:

what system calls find uses

how it interacts with the system

This helps administrators decide:

➡ which system calls should be monitored with audit rules.

Linux Firewalls — Easy Explanation
What is a Firewall?

A firewall is a security system that controls network traffic.

It checks incoming and outgoing packets and decides:

✅ Allow the packet

✕ Block (drop) the packet

This decision is based on a ruleset (security rules).

Why Linux is Good for Firewalls

Linux systems can be used to build many firewalls cheaply.

Instead of buying expensive hardware firewalls, organizations can use:

Linux servers

Open-source firewall software

This allows building many firewalls across the network at low cost.

This is part of Defense-in-Depth (multiple security layers).

Two Types of Linux Firewalls

There are two main types:

1 Network-Based Firewall
2 Host-Based Firewall

1 Network-Based Firewalls

A network firewall protects an entire network.

It is placed between networks.

Example:

Internet → Firewall → Internal Network
How It Works

Packet arrives

Firewall inspects packet

Firewall checks ruleset

If rule says deny → packet dropped

If rule says allow → packet routed to another interface

So the packet passes through the firewall.

Example

A common example is pfSense.

It is used as a network gateway firewall.

2 Host-Based Firewalls

A host firewall protects one computer only.

It runs directly on the device.

Example:

Laptop / Server → Host Firewall → Operating System
How It Works

Packet arrives

Firewall checks rules

If deny → packet dropped

If allow → packet sent to the operating system

So the packet goes up the protocol stack.

Example

A common Linux host firewall is:

iptables
Main Difference Between the Two

Both firewalls do almost the same thing:

1 Receive packet
2 Inspect packet
3 Check rules
4 Drop packet if denied

The only difference happens when the packet is allowed.

Firewall Type	What Happens After Packet is Allowed
Network firewall	packet is routed to another interface
Host firewall	packet is sent to the operating system

What is iptables?

iptables is the built-in firewall in the Linux kernel.

It allows administrators to control network traffic by creating rules that:

✅ allow packets

❌ block packets

forward packets

perform NAT (Network Address Translation)

Because it is built into Linux, many systems use iptables instead of expensive commercial firewalls.

How iptables Works

iptables inspects network packets and checks them against rules.

The process is:

Packet arrives
↓
Check firewall rules
↓
Rule match?
↓
YES → apply action (ACCEPT / DROP / REJECT)
NO → check next rule

Packets move through chains of rules.

iptables Chains

A chain is a list of firewall rules.

The main chains are:

Chain	Purpose
INPUT	incoming packets to the system
OUTPUT	packets leaving the system
FORWARD	packets passing through the system

Example command:

sudo ip6tables -L

This shows the firewall rules.

Example output:

Chain INPUT (policy ACCEPT)
Chain FORWARD (policy ACCEPT)
Chain OUTPUT (policy ACCEPT)

If there are no rules, the firewall allows everything by default.

Example Rule Structure

Example rule:

iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Meaning:

Option Meaning
-A INPUT add rule to INPUT chain
-p tcp protocol TCP
--dport 22 destination port 22
-j ACCEPT allow packet

So this rule:

➡ allows SSH traffic (port 22).

Example Basic Firewall Setup
Step 1 — Remove old rules
iptables -F

This flushes all existing rules.

Step 2 — Default policy (block everything)
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP

Now all traffic is blocked by default.

Step 3 — Allow loopback traffic

Loopback is internal communication inside the system.

```
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
```

Step 4 — Allow SSH access

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

This allows remote SSH login.

Step 5 — Allow outgoing connections

```
iptables -A OUTPUT -p tcp -m state --state NEW,ESTABLISHED -j ACCEPT
iptables -A OUTPUT -p udp -m state --state NEW,ESTABLISHED -j ACCEPT
iptables -A OUTPUT -p icmp -m state --state NEW,ESTABLISHED -j ACCEPT
```

This allows the system to send TCP, UDP, and ICMP traffic.

Stateful Firewalling

iptables supports stateful inspection.

This means it tracks connection states.

Example option:

```
-m state --state ESTABLISHED
```

Connection states:

State	Meaning
NEW	new connection
ESTABLISHED	connection already exists
RELATED	related connection
INVALID	unknown or malformed packet

Example rule:

```
iptables -A INPUT -p tcp -m state --state ESTABLISHED -j ACCEPT
```

This allows packets belonging to existing connections.

Network Firewall Example

Example service network:

Servers:

Web server

Mail server

DNS server

Firewall rules might include:

Allow web traffic:

```
iptables -A FORWARD -d web -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -d web -p tcp --dport 443 -j ACCEPT
```

Allow email:

```
iptables -A FORWARD -d mail -p tcp --dport 25 -j ACCEPT
```

Allow DNS:

```
iptables -A FORWARD -d dns -p udp --dport 53 -j ACCEPT
iptables -A FORWARD -d dns -p tcp --dport 53 -j ACCEPT
```

Allow SSH administration:

```
iptables -A FORWARD -s admin -d 10.2.3.0/24 -p tcp --dport 22 -j ACCEPT
```

Block everything else:

```
iptables -A FORWARD -j DROP
```

Important iptables Options

Option	Meaning
-A	append rule
-L	list rules
-p	protocol
--dport	destination port
-j	action
-m state	connection state matching

Actions in iptables

Common actions:

Action	Meaning
ACCEPT	allow packet
DROP	silently discard packet
REJECT	block and notify sender

Why iptables is Powerful

iptables can:

inspect packets

filter traffic

perform NAT

track connection states

protect hosts or networks

But it is complex to configure.

That is why many systems now use higher-level tools like:

firewalld

nftables

What is rkhunter?

rkhunter is a security tool in Linux/UNIX that checks if your system is infected with dangerous malware.

Think of it like a security scanner or antivirus for Linux.

It looks for:

Rootkits → malware that hides deep inside the system

Backdoors → secret ways for hackers to access your computer

Local exploits → programs that try to gain admin (root) privileges

How rkhunter detects problems

rkhunter checks your system in several ways:

1 File hash comparison

It checks important system files and compares them with known safe versions in databases.

Example idea:

Original file hash → ABC123

Your system file hash → XYZ999

If they are different it may mean the file was modified by malware.

2 Searches suspicious places

It scans directories where rootkits usually hide.

Example:

hidden files

suspicious system folders

3 Checks permissions

It checks if files have wrong or dangerous permissions.

Example:

A normal user should not modify system files.

4 Checks kernel modules

It looks for suspicious code in kernel modules (very low-level system components).

Important rkhunter commands
Update database
rkhunter --update

This downloads the latest security signatures.

You should do this regularly.

Create system baseline
rkhunter --propupd

This creates a baseline of clean system files.

Later scans compare files to this baseline.

Run a full scan
rkhunter -c --enable all --disable none

This performs a complete system security check.

View results

Logs are stored in:

/var/log/rkhunter.log

Check this file to see warnings or alerts.

Automatic checks

It is recommended to run rkhunter automatically using:

cron jobs

system timers

This way the system scans regularly without manual work.

What is chkrootkit?

chkrootkit is a Linux security tool used to check if a system contains rootkits.

A rootkit is a type of malware that hides inside the system and gives hackers secret control of the computer.

So chkrootkit = tool that checks for hidden hacker programs.

What does chkrootkit check?

It scans the system to find:

Suspicious processes

Known rootkit files

Hidden malware

Strange system behavior

It looks for signatures (patterns) of known rootkits.

How chkrootkit works

chkrootkit uses normal Linux tools like:

strings

grep

ps

It checks things like:

1 Suspicious programs

It scans system programs to see if they contain rootkit signatures.

2 Hidden processes

It compares two things:

/proc → shows running processes directly from the system

ps command → shows processes using normal tools

If a process appears in /proc but not in ps, it may be hidden by a rootkit.

3 Known malicious files

It searches for files that are commonly used by rootkits.

How to run chkrootkit

The scan command is very simple:

chkrootkit

This starts a full rootkit scan of the system.

False positives (important)

Sometimes the tool reports something suspicious that is actually normal.

Example:

Bind shell infected on port 465

This often happens on mail servers and is usually not a real infection.

So administrators must check alerts carefully.

Important limitation

Rootkits keep improving.

Hackers can design rootkits that hide from detection tools, including:

chkrootkit

rkhunter

This means no tool is perfect, and multiple security checks are recommended.

What are Configuration Management Tools?

Configuration management tools are software used to manage and control many computers at the same time.

They help system administrators make sure all systems are configured the same way.

Think of them like a remote control for many servers.

Why are they important for security?

In big companies there may be hundreds or thousands of Linux servers.

Without these tools:

Each server might be configured differently

Some servers might be less secure

Updating everything would take a lot of time

Configuration tools solve this problem.

They allow administrators to apply security settings to all systems at once.

What can these tools control?

They can manage things like:

Security settings

Installed software packages

System updates

User permissions

Firewall rules

Example:
An administrator can configure 100 servers in one command instead of configuring them one by one.

Similar idea in Windows

In Windows, there is a similar concept called:

Group Policy

It allows administrators to:

enforce security rules

control updates

manage user settings

Configuration management tools do something similar for Linux/UNIX systems.

Types of tools

There are many tools available:

Open-source tools

Free tools used by many companies.

Examples:

Ansible

Puppet

Chef

SaltStack

Commercial tools

Paid tools with extra support and features.

Companies sometimes choose these for enterprise environments.

Simple example

Imagine a company with 200 Linux servers.

The administrator wants to:

disable a service

install a security update

configure a firewall rule

Without configuration tools → must log into 200 servers manually.

With configuration tools → one command updates all servers.

What is Ansible?

Ansible is an open-source automation tool used to manage and configure many servers automatically.

Instead of configuring servers one by one, Ansible allows you to control many machines at once.

Think of it like a remote control for all your servers.

What can Ansible do?

Ansible can automate many system administration tasks, such as:

Installing software

Creating user accounts

Updating servers

Configuring security settings

Setting up servers or virtual machines

It works for:

Cloud servers

Local servers (on-premises)

Main parts of Ansible

1 Control Node

The control node is the main computer where Ansible is installed.

From this machine, the administrator sends commands to other servers.

Example:

Admin computer (Control Node)
↓
Servers

2 Managed Nodes

Managed nodes are the servers or devices that Ansible controls.

Examples:

Linux servers

virtual machines

network devices

These machines receive instructions from the control node.

3 SSH Connection

Ansible communicates with servers using SSH.

Important point:

✅ No agent needs to be installed on servers
It only uses SSH access.

4 Playbooks

Playbooks are files that contain automation tasks.

They describe what you want the system to do.

Playbooks are written in a simple language called YAML.

Example playbook idea:

Install nginx
Create user
Start service

The same playbook can run on many servers at the same time.

5 Inventory

The inventory is a file that lists all servers Ansible manages.

Example:

webservers
server1
server2
server3

Then Ansible can run a playbook on all servers in that group.

Simple Example

Imagine a company with 50 servers.

The administrator wants to:

install Apache

create a user

update the system

Without Ansible ❌
Log into 50 servers manually.

With Ansible ✅
Run one playbook → all servers are updated.

What is chroot()?

chroot() is a security feature in UNIX/Linux that isolates an application inside a specific folder.

It changes the root directory of the application.

Normally the root directory is:

/

But with chroot(), the program thinks that another folder is the root.

Simple idea

Imagine a program is placed inside this folder:

/var/app/

If chroot() is used, the program can only see:

/var/app/
/var/app/files
/var/app/logs

It cannot see the rest of the system, like:

/etc
/home
/bin

To the program, /var/app looks like the whole system.

Why is this useful?

It improves security.

If a hacker breaks into the application:

They can only access files inside the chroot directory

They cannot access the rest of the operating system

So the damage is limited.

Example

Imagine a web server is hacked.

Without chroot() ❌
The attacker might access:

/etc/passwd
/system files
/other applications

With chroot() ✅
The attacker can only access files inside something like:

/var/www/
Applications that use chroot()

Some programs already support chroot automatically, such as:

TFTP

FTP (anonymous FTP)

BIND (DNS server)

SSH

These services are often exposed to the internet, so isolation is important.

When applications do not support it

Some applications do not include chroot() internally.

Example:

Apache web server

In this case, administrators can use a chroot wrapper program to run the application inside a restricted directory.

Example with TFTP

TFTP servers often use:

/tftpboot

The server can only send files from this directory, not from the whole system.

Connection to containers

chroot() is considered the early idea behind containers.

Modern technologies like:

Docker

LXC

Linux containers

use more advanced isolation, but the idea started with chroot().

Containers

lightweight

share the host OS kernel

run applications

Virtual Machines

heavy

include a full operating system

run complete virtual computers

What is LXC?

LXC (Linux Containers) is a technology used to create containers in Linux.

A container is a small isolated environment where an application can run safely.

Think of it like a mini-system inside Linux.

It contains everything needed for an application to run.

What LXC does

LXC allows you to:

create containers

run applications in isolated environments

distribute containers easily

run multiple applications safely on one system

All containers run on the same Linux kernel.

Why LXC is useful

LXC is often considered the middle ground between chroot and virtual machines.

1 chroot

isolates only a directory

very limited isolation

2 LXC containers

isolates applications and system resources

lightweight

uses the same kernel

3 Virtual machines

runs a full operating system

very strong isolation

uses more memory and storage

So LXC gives better isolation than chroot, but is lighter than virtual machines.

How LXC creates isolation

LXC uses special Linux kernel features to isolate containers.

Kernel Namespaces

Namespaces separate system resources for each container.

Examples:

PID namespace → separate process list

Network namespace → separate network interfaces

Mount namespace → separate filesystem

User namespace → separate users

IPC namespace → separate communication between processes

This makes each container look like its own system.

CGroups (Control Groups)

CGroups control resources.

They limit things like:

CPU usage

Memory

Disk usage

Example:

Container A → 2 GB RAM
Container B → 1 GB RAM

Security features used by LXC

LXC also uses Linux security systems:

AppArmor → security profiles

SELinux → access control

Seccomp → restricts system calls

Kernel capabilities → limits root privileges

chroot / pivot_root → filesystem isolation

These features help protect containers from each other.

The Core of Containers

Containers work mainly because of two Linux kernel features:

1 cgroups
2 namespaces

These two features make containers secure and isolated.

1 cgroups (Control Groups)

cgroups control how many system resources a process can use.

They were originally developed by Google.

What resources do they control?

Mainly:

CPU

Memory (RAM)

Disk usage

Network usage

What do they do?

They limit and monitor resources for a group of processes.

Example:

Server has 8 GB RAM

You create containers:

Container A → 2 GB RAM

Container B → 1 GB RAM

Container C → 3 GB RAM

Even if one container tries to use more memory, cgroups will stop it.

This prevents one application from using all system resources.

2 Namespaces

Namespaces provide isolation between processes.

They were originally developed by IBM.

They make each application think it is running alone on the system.

What namespaces isolate

Examples:

Processes (PID namespace)

Network interfaces

Filesystem mounts

Users and IDs

Hostnames

Example

Two containers are running on one server.

Each container might think:

Process ID = 1

Even though many processes exist on the real system.

This happens because namespaces hide the rest of the system.

LXC

original Linux containers

behaves like a mini operating system

Docker

newer container system

designed for running single applications

more portable and flexible

Docker containers:

Package apps with all dependencies

Run consistently across different systems

Are isolated and secure

Make deployment fast and easy

What is a Docker Image?

A Docker image is like a blueprint for a container.

It contains:

The file system (all files and libraries needed)

Startup instructions (how to run the application)

A Docker container is just a running instance of a Docker image.

2 Dockerfile

A Dockerfile is a text file that tells Docker how to build an image.

You write instructions like:

Which base system to use (Ubuntu, Alpine, etc.)

What software to install

Configuration files

Commands to run at startup

You can use a public image as a base and add your own packages.

3 Building Images

Docker can build images at runtime from a Dockerfile.

To save time, images can be prebuilt and stored in a registry.

4 Docker Registries

A Docker registry is a place to store and share Docker images.

Types:

Docker Hub (Public)

Public repository: hub.docker.com

Official images from companies like Ubuntu, Nginx, NodeJS

You can also store your own images, but they are public

Private Registries

Hosted privately or by cloud providers

Only authorized users can access your images

Useful for storing custom company images

What is Docker Swarm Mode?

Docker Swarm Mode is Docker's built-in container orchestration tool.

It allows you to manage multiple Docker hosts (computers running Docker) as a single cluster.

It helps deploy, manage, and scale containers efficiently in a production environment.

Key Concepts

Nodes

Machines in the swarm are called nodes.

Two types of nodes:

Manager nodes: Control the swarm, handle scheduling and cluster management.

Worker nodes: Run containers assigned by managers.

Services

A service defines how a containerized application should run.

Includes:

Number of replicas

Networking

Update policies

Desired State

You declare how your service should run (number of replicas, etc.).

Docker Swarm ensures the cluster matches that state automatically.

Why Use Docker Swarm?

Built into Docker – no extra software needed.

High availability – managers can handle failures and recover automatically.

Scaling – add/remove containers easily across multiple nodes.

Simpler than Kubernetes – easier to set up for small-to-medium deployments.

What is Kubernetes?

Kubernetes (k8s) is an open-source container orchestration platform.

It automates:

Application deployment

Scaling of containers

Management and updates

Originally designed by Google (based on their internal system “Borg”) and now maintained by the Cloud Native Computing Foundation.

Key Concepts

Cluster

A Kubernetes cluster runs on multiple nodes (machines).

Ensures high availability: if a node fails, workloads automatically move to healthy nodes.

Pods

Pods are the smallest deployable units in Kubernetes.

Each pod can contain one or more containers that share networking and storage.

Components

Kubernetes is made of many internal components:

API Server: Interface to interact with the cluster

Controller Manager: Ensures the desired state of resources

Scheduler: Assigns pods to nodes

etcd: Key-value store for cluster data

These components communicate via the Kubernetes API, making it highly extensible.

Desired State Management

You declare how your applications should run (number of replicas, resources, etc.).

Kubernetes continuously works to maintain that state, automatically replacing failed pods.

Docker Security Best Practices

To minimize risks and attacks on Docker containers, follow these key practices:

Use minimal and trusted base images

Choose official or verified images.

Fewer libraries = smaller attack surface.

Avoid running containers as root

Create a dedicated user inside the container with minimal permissions.

Reduces the impact if the container is compromised.

Sign and verify images

Use cryptographic signing to ensure the image hasn't been tampered with.

Prevents man-in-the-middle (MITM) attacks.

Monitor images for vulnerabilities

Regularly check custom or base images for known security flaws.

Never store secrets in images

Avoid putting API keys, passwords, or connection strings directly inside images.

Use Docker secrets or environment variables securely.

Do not share the host network

Avoid --net=host unless necessary.

Sharing the host network can expose the host to container attacks.

Disable unused services and ports

Don’t enable SSH inside containers or expose ports you don’t use.

Avoid privileged mode

Do not run containers with the --privileged flag.

It gives containers full access to the host, which is dangerous.

Vulnerability Management in Docker Images

Docker images, like operating systems, can contain software packages with known vulnerabilities. To keep your images secure, you can use open-source scanning tools to detect and manage these vulnerabilities.

Key Tools Mentioned

Clair

Type: API-driven static container analysis tool

Features:

Scans Docker images using multiple CVE databases (like Debian Security Bug Tracker, Ubuntu CVE Tracker, Red Hat Security Data)

Built by CoreOS (part of Red Hat)

Integrated into Quay.io, an alternative container registry to Docker Hub

Anchore

Type: Command-line tool or Docker container for CVE-based reporting

Features:

Reports security vulnerabilities based on known CVEs

Allows custom security policies using allowlists/denylists, credentials, or file contents

Can run standalone or inside a Docker container

Why It Matters

Docker containers virtualize the kernel but still have a full filesystem.

This means they can inherit vulnerabilities from standard Linux packages included in the image.

Using tools like Clair or Anchore makes it easier to detect, report, and manage vulnerabilities in custom Docker images.

AWS Identity and Access Management (IAM)
Purpose

Control and restrict access to AWS resources.

Acts as a security perimeter since IAM users are exposed to the Internet.

Why It’s Important

Improper IAM configuration is a major cloud security risk.

Can lead to unauthorized access and data leaks.

Users must be authenticated (prove identity) and authorized (permission to act) before accessing resources.

Key Features

Access Management

Grant access to use or administer AWS resources.

Granular Permissions

Assign precise permissions for different resources.

Ensures only authorized users can perform specific actions.

Integration with AWS Services

Implement policies, temporary access, and service roles.

Identity Federation

Allow external users to access AWS without creating IAM accounts.

Access Analysis

Monitor and analyze permissions and access patterns.

Core Components

Users – Individual identities with specific permissions.

Groups – Collection of users with shared permissions.

Policies – JSON documents defining allowed or denied actions.

Roles – Assignable sets of permissions for AWS services or temporary access.

AWS IAM Key Concepts

1. AWS Account

Strong isolation barrier – One account cannot access another by default.

Administrative capabilities – Full control over resources and services.

Resource container – Holds all AWS resources you deploy.

Root user – Initial user created at account creation:

Full access to all resources and settings (including billing).

Cannot be deleted or restricted directly.

Critical: compromise of the root user has huge security impact.

Isolated IAM – IAM users and roles are account-specific.

Billing & Cost Management – Centralized in the account.

2. Principals

Definition: Any entity (person, app, or service) that requests an action on AWS resources.

Can be:

Root user

IAM user

Application or service

Role

3. Users

Represents a person or application with long-term credentials.

Examples: AWS Console login password or CLI/SDK access keys.

Default permissions: None – users must be explicitly granted permissions.

Organization: Users can be grouped into groups for easier permission management.

4. Roles

Identity with specific permissions but no long-term credentials.

Can be assumed temporarily by anyone who needs it.

Use cases:

IAM users in the same AWS account

IAM users in a different AWS account

AWS services (e.g., EC2 instances)

External users authenticated via external identity providers

5. Policies

Define who can do what on a resource, optionally with conditions.

Types of policies:

Managed Policies:

Can be AWS-managed or customer-managed.

Reusable across multiple users, groups, or roles.

Inline Policies:

Embedded directly into a single user, group, or role.

Policies attach permissions to users, groups, or roles to control access.

Network ACL (NACL)

Stateless firewall → It does not remember connections.

Works at the Subnet level → Protects the whole subnet.

Allow OR Deny rules → You can explicitly block traffic.

Example

If you allow HTTP (port 80) inbound:

You must also allow outbound response ports (because NACL is stateless).

Otherwise, the response traffic will be blocked.

Simple idea

Think of NACL like a security guard at the neighborhood gate

It controls traffic entering or leaving the entire subnet area.

It checks every request and response separately.

Default NACL

Allows all traffic in and out by default.

2 Security Group (SG)

Stateful firewall → It remembers connections.

Works at the Instance level → Protects each EC2 instance.

Only Allow rules → You cannot create explicit deny rules.

Example

If you allow HTTP (port 80) inbound:

The response traffic is automatically allowed.

You do not need extra outbound rules.

Simple idea

Think of Security Group like a security guard at the house door

It protects each EC2 instance individually.

Once someone is allowed in, their responses are automatically allowed.

Default Security Group

Allows traffic only from other instances in the same security group

AWS Network Firewall is an advanced firewall service from AWS that protects your entire VPC network from attacks and malicious traffic.

Think of it as a powerful security system for your cloud network.

What AWS Network Firewall Does

Managed by AWS → AWS handles setup, scaling, and availability.

Highly available → Works even if part of the system fails.

Protects the whole VPC → Filters traffic going in and out of the network.

Security Features

1 Stateful Inspection

The firewall remembers connections.

It understands requests and responses.

Example:
If a computer sends a request, the firewall knows the response is valid.

2 Intrusion Detection & Prevention (IDS / IPS)

Detects suspicious or malicious activity.

Can block attacks automatically.

Examples:

Port scans

Malware communication

Exploit attempts

3 Web Filtering

You can block dangerous or unwanted websites.

Example:

Block malicious domains

Block certain categories of websites

4 Custom Security Rules

You can create rules to:

Allow trusted traffic

Block suspicious IP addresses

Prevent unauthorized access

Logging and Monitoring

AWS Network Firewall can send logs to AWS services like:

Amazon CloudWatch

Amazon S3

Amazon Kinesis

These logs can also be sent to SIEM systems (security monitoring tools) for deeper analysis.

Example SIEM tools:

Splunk

IBM QRadar

Elastic Security

AWS Shield is a service that protects AWS applications from DDoS attacks.

What is a DDoS attack?

A Distributed Denial of Service (DDoS) attack happens when attackers send huge amounts of traffic to a server to make it slow or completely unavailable.

Example:
Millions of fake requests are sent to your website so real users cannot access it.

What AWS Shield Does

Detects suspicious traffic

Automatically blocks or mitigates attacks

Keeps your application online

AWS Services it Protects

AWS Shield works with:

Amazon EC2

Elastic Load Balancer (ELB)

Amazon CloudFront

Amazon Route 53

AWS Global Accelerator

Two Versions

1 Shield Standard (Free)

Automatically enabled for all AWS users

Protects against common DDoS attacks

2 Shield Advanced (Paid)

Protection against large and complex DDoS attacks

Detailed attack reports

AWS security team support

It filters HTTP/HTTPS requests before they reach your application.

Examples of Attacks it Stops

SQL Injection

Cross-Site Scripting (XSS)

Bots or malicious requests

Exploits in web applications

OWASP Top 10 Protection

AWS WAF has managed rules that protect against the OWASP Top 10, which are the most common web security risks.

Works With

Elastic Load Balancer (ELB)

Amazon CloudFront

Amazon API Gateway

Custom Rules

You can create rules like:

Block specific IP addresses

Block certain countries

Limit request rates

Filter malicious patterns

AWS Firewall Manager

If a company has many AWS accounts, AWS Firewall Manager helps manage security rules centrally.

It can enforce across all accounts:

AWS WAF rules

Security Groups

AWS Shield protections

hat Amazon Macie Does
Discover Sensitive Data

Macie scans your S3 buckets to find sensitive information such as:

PII (Personally Identifiable Information)

Names

Email addresses

Phone numbers

Passport numbers

Financial data

Confidential company data

It uses:

Machine learning

Pattern matching

to detect this information automatically.

2 Give Visibility

Macie creates a dashboard showing:

Which S3 buckets contain sensitive data

Which buckets are public

Which buckets are not encrypted

This helps security teams see risks quickly.

3 Alerts and Findings

If Macie finds sensitive data, it will generate alerts called findings.

Example alerts:

Sensitive data found in a public S3 bucket

Unencrypted sensitive data

Unexpected data exposure

4 Automatic Response

After detecting a problem, you can automate actions using AWS services like:

AWS Lambda

AWS Step Functions

Example actions:

Automatically block public access

Encrypt the bucket

Send security notifications

5 Compliance Support

Amazon Macie helps organizations comply with security regulations like:

GDPR (European data protection law)

HIPAA (healthcare data protection)

Because it helps detect exposed personal data.

Simple Example

Imagine a company stores files in Amazon S3.

One file contains:

Customer names

Email addresses

Credit card data

Macie scans the bucket and:

Detects sensitive information

Creates an alert

Security team fixes the issue

AWS KMS is a service that creates and manages encryption keys used to protect data across AWS services.

Why use multiple accounts?

Different environments should be separated.

Example:

Account	Purpose
Dev Account	Developers testing code
Test Account	Testing environment
Production Account	Real users and applications

This prevents mistakes like developers accidentally breaking production systems.

Organizational Units (OUs)

Accounts can be grouped into OUs (Organizational Units).

Example structure:

```
Root
 ├── Production OU
 |    ├── Prod Account 1
 |    └── Prod Account 2
 └── Development OU
      ├── Dev Account
      └── Test Account
```

Service Control Policies (SCP)

SCPs define maximum permissions for accounts.

Example rule:

Block deleting S3 buckets

Prevent disabling CloudTrail

Restrict some AWS services

Important:

SCPs apply to all users in that account (even root).

2 Least Privilege Principle

The least privilege principle means:

A user should only have the minimum permissions needed to do their job.

Example:

❌ Bad practice
Developer gets full admin access

✅ Good practice
Developer only gets permissions like:

Read logs

Deploy applications

Access development resources

Types of IAM Policies
Identity-Based Policies

Attached to:

Users

Groups

Roles

Example:

User → Allowed to read S3 bucket
Resource-Based Policies

Attached directly to resources.

Example:

S3 Bucket Policy → Allow another account to access bucket
Permission Boundaries

Permission boundaries define maximum permissions an identity can have.

Example:

Developer role policy allows:

EC2:StartInstances
EC2:StopInstances

But permission boundary says:

Only EC2 actions allowed

Even if someone tries to add S3 admin permission, it will not work.

This prevents privilege escalation attacks.

3 Secure Resource Sharing

Sometimes you need to share resources between AWS accounts.

Example:

Share S3 bucket

Share VPC resources

Share subnets

There are several methods.

Assume Role

Another account can temporarily assume a role in your account.

Example:

Account A → Assume role → Access resources in Account B

This is a secure cross-account access method.

AWS Resource Access Manager (RAM)
4

AWS RAM allows you to share resources across accounts in AWS Organizations.

Example resources you can share:

VPC subnets

Transit Gateway

Route53 Resolver rules

Access is automatically updated if accounts move between OUs.

IAM Access Analyzer

IAM Access Analyzer checks who can access your resources.

It detects:

Public access

Cross-account access

Unexpected permissions

Example finding:

S3 bucket is publicly accessible

This helps identify security risks quickly.

Simple Summary

Concept	Purpose
AWS Organizations	Manage multiple accounts
Organizational Units	Group accounts
SCP	Restrict maximum permissions
Least Privilege	Give minimal access
Identity Policies	Attached to users/roles
Resource Policies	Attached to resources
Permission Boundaries	Limit maximum permissions
AWS RAM	Share resources between accounts
IAM Access Analyzer	Detect risky access

The goal is to protect AWS systems using multiple security layers (defense in depth).

1 Network Isolation

Use VPCs and subnets to separate resources.

Create public and private zones.

Put sensitive systems (servers, databases) in private subnets.

2 Control Network Traffic
Use AWS network security tools:

Security Groups → stateful firewall for EC2 instances

NACLs (Network ACLs) → stateless firewall for subnets

Route Tables → control where traffic goes

3 Do Not Expose Servers Directly

EC2 instances should not be directly accessible from the Internet.

Use Load Balancer or CloudFront in front of them.

4 Inspect Traffic
Use security services to analyze traffic:

AWS WAF → protects web apps from attacks (SQL injection, XSS)

AWS Network Firewall → inspects network traffic

AWS Transit Gateway → centralizes traffic between networks

5 Centralized Security
Large systems often send traffic to an Inspection VPC where a firewall checks it before reaching servers.

Vulnerability Management

Regularly scan systems and fix vulnerabilities.

Examples:

Patch systems to fix security issues

Scan for CVEs (Common Vulnerabilities and Exposures)

Check libraries and dependencies for vulnerabilities

Use tools like Amazon Inspector or ECR image scanning

Another technique:

Fuzzing → testing applications by sending invalid or random inputs to find bugs.

2 Minimize the Attack Surface

Use fewer components, libraries, and services.

Why?

More software = more possible vulnerabilities.

Good practice:

Install only necessary software

Follow CIS security benchmarks for system hardening.

3 Avoid Direct Human Access

Humans should not directly manage servers.

Instead use automation tools, such as:

AWS CloudFormation → automatic infrastructure deployment

AWS Systems Manager → automate maintenance tasks

This reduces human errors and misconfigurations.

4 Use Managed AWS Services

Use services where AWS manages the infrastructure.

Examples:

Amazon RDS → managed database

AWS Lambda → serverless compute

Amazon ECS → container management

Benefits:

Automatic patching

Less manual work

Better security

5 Verify Software Integrity

Make sure software comes from trusted sources.

Methods:

Code signing certificates

Hash verification

This ensures the software was not modified or tampered with.

6 Automate Security

Automate security processes like:

patching

scanning

deployment

monitoring

Automation helps reduce human mistakes and improves security.

Terraform is a tool that lets you create and configure cloud infrastructure automatically using code instead of doing it manually.

Why is this useful?

It reduces human mistakes ✕

It keeps systems secure by default

It makes deployments repeatable and consistent

This approach is called Infrastructure as Code (IaC).

1 Provider File (provider.tf)

This file tells Terraform which cloud provider to use and where to deploy resources.

Example:

```
provider "aws" {
  region     = "us-west-2"
  access_key = "my-access-key"
  secret_key = "my-secret-key"
}
```

What it means

provider "aws" → we are using AWS cloud.

region = us-west-2 → resources will be created in this AWS region.

access_key / secret_key → credentials used to access AWS.

Important security note:
Hardcoding keys like this is not recommended, because someone could steal them.

2 Main File (main.tf)

This file defines what resource we want to create.

Example:

```
resource "aws_instance" "sans_ec2_instance" {
  name = "sans_ec2_instance"
  ami = "ami-005e54dee72cc1d00"
  instance_type = "t2.micro"

  network_interface { ... }

  instance_initiated_shutdown_behavior = "terminate"

  monitoring {
    enabled = true
  }

  security_group = "sg-12345678"
}
```

What this code does

It creates an AWS EC2 virtual machine with specific settings:

name → the server name.

AMI → the operating system image used to start the server.

instance_type → server size (CPU & RAM).

network_interface → networking settings.

shutdown behavior

terminate → deleting the server when shutdown.

monitoring

enables detailed monitoring.

security_group

firewall rules for the server.

3 Why Automation Is Important

Instead of someone manually clicking in AWS console:

Write configuration once

Run Terraform

Infrastructure is created automatically

Benefits:

Less human errors

Same configuration every time

Faster deployment

More secure defaults

Example:
You can force every server to:

only have private IP

block internet traffic

use specific security groups

AWS CloudFormation is a tool that lets you create AWS infrastructure using code templates instead of manual clicks in the AWS console.

This is also called Infrastructure as Code (IaC).

So instead of manually creating servers, networks, and security settings, you write a template and AWS builds everything automatically.

1 What CloudFormation Does

CloudFormation allows you to:

Define infrastructure using templates

Create AWS resources automatically

Keep configuration consistent

Reduce human errors

Example resources you can create:

EC2 servers

VPC networks

Security groups

Databases

Load balancers

All from one template file.

2 Stack Concept

In CloudFormation, resources are deployed as a Stack.

Think of a stack like a package of resources created together.

Example stack:

1 EC2 server

1 database

1 security group

1 load balancer

CloudFormation creates them in the correct order automatically.

3 Automating Security Best Practices

A big advantage is that security rules can be written directly in the template.

Example security best practices you can enforce:

Servers cannot have public IPs

Only specific ports are open

Encryption is enabled

Logging and monitoring are enabled

Because the template contains these rules, every deployment is automatically secure.

No one can forget security settings.

4 Faster and Consistent Deployments

Without CloudFormation:

Engineer logs into AWS

Creates resources manually

Configures security manually

This can cause mistakes and inconsistencies.

With CloudFormation:

Write template once

Deploy stack

AWS builds everything automatically

So every environment (dev, test, production) looks exactly the same.

5 CloudFormation StackSets

StackSets allow you to deploy the same infrastructure across:

Multiple AWS accounts

Multiple AWS regions

Example:

Deploy the same security configuration to all company AWS accounts automatically.

6 CloudFormation Registry Extensions

CloudFormation also has extensions in a registry.

These extensions can add tools for:

- Monitoring
- Incident management
- Version control

This helps teams track changes and monitor infrastructure over time.

Type	Meaning	Protection
Data at Rest	Stored data	Encryption + Tokenization
Data in Transit	Data moving on network	TLS / HTTPS

Even if we already have security protections, attacks can still happen.
So organizations must have a plan to detect and respond to security incidents quickly.

A security incident could be things like:

Unauthorized login

Data breach

Malware infection

Someone disabling security logs

The goal is to detect the problem, contain it, and restore systems safely.

Why Incident Response Is Important

Security controls alone are not enough.

Organizations must be ready to:

Detect attacks

Investigate what happened

Stop the attack

Recover systems

Learn from the incident

Guidelines for this process come from the NIST SP 800-61 Incident Handling Guide.

Incident Response Steps in AWS

AWS divides incident response preparation into 4 main approaches.

1 Educate

Teams must learn cloud technologies and security tools.

This includes:

Training on AWS services

Learning security tools

Practicing incident response

Example activities:

Security workshops

Hands-on labs

Security incident exercises

Goal:

Ensure teams know how to react during an attack.

2 Prepare

Organizations must prepare before incidents happen.

Preparation includes:

Creating an incident response plan

Defining roles and responsibilities

Ensuring responders have the right permissions

Making sure logs and evidence are collected

Also create runbooks / playbooks.

Example playbook:

If a server is compromised:

Isolate the server

Collect logs

Take a snapshot

Investigate

Restore system

This helps teams react quickly instead of improvising.

3 Simulate

Organizations should practice security incidents.

This is called:

SIRS — Security Incident Response Simulations

These simulations:

Use realistic attack scenarios

Train responders

Test procedures

Improve confidence during real attacks

Example scenario:

A user account is compromised

Security team must detect and stop it

This helps find weaknesses in the response process.

4 Iterate (Improve & Automate)

After learning from incidents, organizations should improve and automate the response.

Instead of relying only on humans, systems can respond automatically.

Example in AWS:

A security event happens

AWS detects it

An automated response is triggered

Example workflow:

CloudTrail event → EventBridge → Lambda function

Example use case:

If someone disables logging (CloudTrail):

EventBridge detects the change

It triggers a Lambda function

Lambda automatically re-enables CloudTrail

This allows faster response without human delay.

What is macOS? (Simple Explanation)

macOS is the operating system made by Apple for their computers such as MacBook Pro, MacBook Air, and iMac.

Before 2016, it was called Mac OS X, but Apple changed the name to macOS to match their other systems:

iOS → iPhone

watchOS → Apple Watch

tvOS → Apple TV

macOS → Mac computers

1 Built on the XNU Kernel

The core of macOS is called the XNU kernel.

The kernel is the central part of an operating system that manages:

CPU

Memory

Hardware

Processes

Because macOS uses this kernel, it also includes many UNIX features.

This means you can use terminal commands similar to Linux/Unix.

Example terminal command:

ls

Shows files in a folder.

2 Darwin Subsystem

At the core of macOS there is an open-source system called Darwin.

Darwin provides:

UNIX foundation

File system management

Security permissions

Networking

Darwin also uses Linux/Unix-like file permissions.

Example permission format:

-rwxr-xr-x

Meaning:

Owner permissions

Group permissions

Others permissions

This is very similar to Linux systems.

3 Apple Interface (Aqua + Finder)

On top of Darwin, Apple adds its graphical interface.

Important components:

Aqua Interface

The graphical design of macOS (windows, buttons, menus)

Finder

The file manager used to browse files and folders.

Example:

Like File Explorer in Windows.

4 Installing Applications in macOS

There are two main ways to install apps.

App Store

Apple’s official store where apps are verified and safe.

Example apps:

Safari extensions

Productivity apps

Games

DMG Files (.dmg)

macOS applications often come as DMG files.

DMG means:

Disk Image File

Example:

app_installer.dmg

A DMG file works like a virtual disk.

When you open it:

macOS mounts it like a drive

You see the installer

You drag the app into the Applications folder

5 Development with Xcode

Developers create macOS and iOS applications using Xcode.

Xcode is Apple's official development environment (IDE).

It includes tools to:

Write code

Compile programs

Debug software

Build apps

Languages supported include:

C

C++

Swift

Objective-C

Python

Ruby

Java

AppleScript

macOS Security Features (Simple Explanation)

macOS includes many built-in security features to protect the computer, user data, and applications.
These protections help defend against malware, phishing, data theft, and unauthorized access.

1 Privacy Controls

macOS allows users to control which apps can access sensitive information.

Apps must request permission before accessing things like:

Microphone

Webcam

Files and folders

Photos

Calendar

Contacts

Location

Example:
If an app wants to use the camera, macOS will show a permission request.

The user can:

Allow access

Deny access

Extra protection

Since macOS Catalina, system files are stored on a read-only partition.

This means:

Malware cannot modify critical system files

Only official updates can change them

2 Keychain (Password Manager)

Keychain is Apple’s built-in tool for storing sensitive information securely.

It can store:

Passwords

Credit cards

Wi-Fi credentials

Certificates

Private keys

The data is encrypted and protected.

Keychain can also sync across devices using iCloud Keychain.

Example:

Save password on Mac

Automatically available on iPhone

3 Automatic Strong Passwords

When creating an account on a website using Safari, macOS can:

Generate a strong random password

Save it automatically in Keychain

Example generated password:

v8T$9kLp!3@q

This helps prevent:

Weak passwords

Password reuse

4 Automatic Updates

New vulnerabilities are discovered frequently.

macOS automatically:

Checks for updates daily

Installs critical security updates

Prompts the user if a restart is required

This reduces the risk of unpatched vulnerabilities.

5 Gatekeeper

Gatekeeper protects the system from malicious applications.

Before an application runs, macOS checks:

If the app is signed by a trusted Apple developer certificate

If the app is not signed:

macOS blocks it by default.

Users can still manually allow it if they trust it.

6 Anti-Phishing & Download Protection

The Safari browser includes protection against:

Phishing websites

Malware websites

Dangerous downloads

Safari uses a denylist from Google Safe Browsing.

If a user visits a malicious site:

Safari displays a warning and blocks access.

7 XProtect (Built-in Anti-Malware)

XProtect is Apple’s built-in malware detection system.

It works by checking files against a database of known malware signatures.

When an app is opened:

1 macOS scans the file
2 If malware is detected → execution is blocked

There is also a Malware Removal Tool that removes known malware automatically.

Note:
XProtect is not a full antivirus, but a basic protection system.

8 Find My (Device Tracking)

If a Mac is lost or stolen, Apple provides Find My.

It allows users to:

Locate the device

Lock the device remotely

Erase the data

Play a sound to locate it nearby

The location is determined using:

IP address

Wi-Fi networks

Bluetooth mesh network (other Apple devices)

9 macOS Firewall

macOS includes a built-in firewall.

The firewall:

Blocks unauthorized incoming network connections

By default:

Apple system apps are allowed

Other apps must be manually approved

There is also stealth mode:

The system ignores ICMP ping requests

Makes the computer harder to detect on a network

Note:
macOS firewall mainly controls incoming connections, not outgoing ones.

FileVault (Disk Encryption)

FileVault encrypts the entire disk.

If the Mac is stolen:

Data remains encrypted

Attackers cannot read the files

Encryption happens automatically in the background after setup.

1 1 Sandboxing & Runtime Protection

Applications from the Mac App Store run in a sandbox.

Sandboxing means:

Apps run in an isolated environment

They cannot access the whole system

Example:

A photo app cannot automatically access:

system files

contacts

other apps' data

Runtime Security Protections

macOS also uses advanced protections:

NOEXEC

Marks memory regions as non-executable

ASLR (Address Space Layout Randomization)

Randomizes memory locations

These protections make exploiting vulnerabilities much harder.

1 2 Apple Security Chip (T2 Chip)

Modern Macs include a T2 Security Chip.

This chip has a Secure Enclave, which performs sensitive operations safely.

It handles:

Disk encryption

Secure boot

Touch ID authentication

Cryptographic operations

Even if malware gets root access, it cannot access the secure enclave.

Even though macOS already has many built-in security features, it is still important to harden the system by applying additional security practices.

Hardening means configuring the system to reduce security risks.

Below are some simple ways to secure a macOS system.

1 Turn On the Firewall

macOS has a built-in firewall that controls incoming network connections.

The firewall helps:

Block unauthorized access

Protect services running on the system

Prevent attackers from connecting to open ports

Example:

If a malicious user tries to connect to your computer over the network, the firewall can block the connection.

However, remember:

The best way to secure a service is to disable it if it is not needed.

2 Turn Off Unnecessary Services

Services running on a computer create potential attack points.

Examples of services:

File sharing

Remote login

Screen sharing

Printer sharing

If a service is not required:

❌ Disable it

This reduces the attack surface of the system.

3 Limit Service Sharing

If services must be used (for example file sharing), they should be restricted.

Good practices include:

Only allow trusted users

Restrict network access

Disable public sharing

Example:

Instead of sharing files with everyone, share them only with specific users.

4 Use Secure File Sharing

If file sharing is required, it should be configured securely.

This means:

Using authentication

Restricting permissions

Encrypting connections when possible

Example:

Allow only specific users to access shared folders.

5 Monitor Access Lists

Access lists define who can access resources.

Examples include:

User permissions

File permissions

Shared folder access

These lists should be reviewed regularly.

Why?

Because:

Employees change roles

Users leave organizations

Permissions may become unnecessary

So access should always follow the least privilege principle.

6 Use Strong Passwords

macOS includes a Password Assistant to help create strong passwords.

It can:

Generate strong passwords

Evaluate password strength

Suggest better passwords

Password strength indicators:

Red → weak password

Yellow → medium strength

Green → strong password

Strong passwords help prevent:

Brute force attacks

Credential theft

Unauthorized access

Vulnerabilities in macOS (Easy Explanation)

Even though macOS is secure, like any software it can still contain vulnerabilities (bugs).
Attackers can exploit these bugs to gain access, bypass security, or run malicious software.

Two examples of macOS vulnerabilities are BuggyCow and Gatekeeper Bypass.

1 BuggyCow Vulnerability

BuggyCow is a privilege escalation vulnerability in macOS.

It was discovered in 2018 by Google Project Zero.

What is Privilege Escalation?

Privilege escalation means:

A normal user gains administrator/root privileges.

Example:

A normal user process → becomes a root process

Why BuggyCow Happens

macOS uses a technique called Copy-on-Write (CoW).

Copy-on-Write means:

When two processes share data

A copy is created when one process tries to modify the data

This prevents programs from changing shared data.

The Problem

BuggyCow exploited a bug in the XNU kernel memory management.

Attackers could:

Mount a disk image

Modify shared data directly

Bypass the Copy-on-Write protection

Result:

An unprivileged process modifies privileged process data

Code runs with higher privileges

This can allow attackers to take control of the system.

2 Gatekeeper Bypass Vulnerability

This vulnerability allows attackers to run malicious applications without security checks.

Normally, macOS uses Gatekeeper to verify applications before running them.

Gatekeeper checks:

Developer signature

Application origin

Malware scanning

But attackers found ways to bypass it.

How the Attack Works

The attack uses two weaknesses:

1 Symlinks inside ZIP files are not checked

A symlink is a shortcut pointing to another file or location.

Example:

file1 → /external-server/malware

2 Automatic network share mounting

When macOS accesses a path like:

/net/server/

The system automatically mounts the network share.

Attack Scenario

The attacker:

1 Creates a ZIP file
2 Adds a symlink pointing to a malicious server
3 Sends the ZIP file to the victim

When the victim:

Opens the ZIP

Clicks the symlink

macOS:

Automatically mounts the attacker's server

Trusts the network share

Runs the application without Gatekeeper verification

Result:

Malicious software executes without security checks.

Malware on macOS

Even though macOS has fewer viruses than Windows, it still has malware.

Two well-known macOS malware examples are FlashBack and CrescentCore.

1 FlashBack Trojan

FlashBack is a trojan malware discovered in 2011.

It pretended to be:

Adobe Flash Player installer

But when installed, it actually:

Installed a backdoor

Connected the system to a botnet

Later versions used a Java vulnerability to infect users automatically.

Impact

In 2012, FlashBack infected:

over 600,000 Mac computers worldwide

2 CrescentCore Trojan

CrescentCore is another macOS trojan discovered in 2019.

It also pretended to be:

Flash Player installer

It was distributed on websites offering free pirated content.

What CrescentCore Does

Once installed:

Creates a backdoor

Allows attackers to control the system

It also tries to avoid detection.

Before running, it checks:

If it is running in a virtual machine

If antivirus software is installed

If detected:

❌ The malware stops running.

This makes it harder for researchers to analyze it.

Important Security Lesson

macOS security tools like:

Gatekeeper

XProtect

are helpful, but not perfect.

Because of this, it is recommended to also use:

Anti-malware software

Security updates

Safe browsing practices